

Linux pour l'admin cloud

Jour 4 — Ouvrir, sécuriser, déployer

Bloc CL-LINUX (jour 9 du cursus) — aujourd'hui, StockLine part en production

Objectifs de la journée

À la fin de cette journée, vous serez capable de :

- Expliquer le **modèle client/serveur de SSH** et l'authentification **par paire de clés**.
- Créer une clé **ed25519**, l'installer sur un serveur (`ssh-copy-id`, `authorized_keys`) et configurer votre client (`~/.ssh/config`, `ssh-agent`).
- Transférer des fichiers avec **scp** et surtout **rsync**, et monter un **tunnel SSH**.
- **Durcir sshd** (drop-ins, `PermitRootLogin no`, `PasswordAuthentication no`) sans vous enfermer dehors, et activer le pare-feu **ufw** dans le bon ordre.
- Lire l'état de santé d'un serveur : `uptime`, `free -h`, `vmstat`, `df -h`.
- **Déployer StockLine** de bout en bout : `rsync` → `venv` → `systemd` → `nginx` → sauvegarde → `ufw`.

Ce déploiement, c'est **mot pour mot le CL-TP1 noté** — puis l'user data de vos instances.

Plan de la journée

1. **SSH en profondeur** — clés, config client, scp/rsync, tunnels (~1 h 30) + ✎ exercice 4-1.
2. **Durcissement** — sshd_config, ufw, fail2ban en démo (~1 h 15) + ✎ exercice 4-2.
3. **Supervision de base** — uptime, mémoire, vmstat, iostat, df (~45 min).
4. **Fil rouge : déployer StockLine** — 💻 démo 4-1 puis 🖋 mini-TP en autonomie (~2 h 30).
5. **Récap du bloc + quiz** + cap sur CL-RÉSEAU (~30 min).

Fil conducteur du cursus : **automatiser, superviser, sécuriser** — aujourd'hui, les trois à la fois, sur une vraie mise en production.

1. SSH en profondeur

La porte d'entrée de tous vos serveurs

SSH : le modèle client/serveur

SSH (Secure SHell) = un canal **chiffré** entre votre poste et un serveur, pour ouvrir un shell, exécuter des commandes, copier des fichiers.

- Côté serveur : le démon **sshd** écoute sur le **port 22/tcp** (service **ssh** sur Ubuntu, déjà actif sur votre VM Multipass).
- Côté poste : le client **ssh** (présent sur macOS, Linux, Windows 10+).

```
ssh ubuntu@192.168.64.10      # utilisateur@IP (multipass info srv-linux)
systemctl status ssh          # sur la VM : le serveur écoute
ss -tlnp | grep :22           # réflexe du J3 : qui écoute sur 22 ?
```

Dans ce deck, **192.168.64.10** désigne l'IP de **votre** VM : relevez la vôtre avec `multipass info srv-linux` avant de rejouer les commandes.

100 % de l'administration cloud passe par là : pas d'écran ni de clavier branché sur une EC2 — SSH est la seule console.

Authentification par clés : qui détient quoi ?

```
POSTE (client)                VM / EC2 (serveur sshd)
~/.ssh/id_ed25519              ~/.ssh/authorized_keys
(clé PRIVÉE, ne sort          (liste de clés PUBLIQUES
JAMAIS du poste)              autorisées pour cet user)
~/.ssh/id_ed25519.pub  — copie ———> une ligne ajoutée ici
```

Connexion :

1. sshd envoie un défi aléatoire
2. le client le SIGNE avec la clé privée
3. sshd vérifie la signature avec la clé publique -> accès
=> aucun secret ne transite sur le réseau

La clé **publique** se distribue sans risque ;
la clé **privée** ne quitte jamais votre poste.

Créer sa paire de clés : `ssh-keygen -t ed25519`

```
ssh-keygen -t ed25519 -C "prenom@utopios-formation"  
# Enter file... : Entrée (défaut ~/.ssh/id_ed25519)  
# Enter passphrase : une phrase de passe (recommandé)  
ls -l ~/.ssh/id_ed25519 ~/.ssh/id_ed25519.pub
```

- **ed25519** : l'algorithme moderne — clés courtes, rapides, sûres (préférez-le à RSA pour toute nouvelle clé).
- La **passphrase** chiffre la clé privée **sur le disque** : un poste volé ne donne pas accès à vos serveurs (`ssh-agent` évitera de la retaper).
- **-C** ajoute un commentaire : identifiez **qui** et **quel poste**.

Pourquoi la clé plutôt que le mot de passe ? Impossible à deviner par force brute, aucun secret rejouable qui circule, révocable individuellement (une ligne à supprimer), et indispensable pour **automatiser** — scripts, rsync, CI.

Rappel cloud : **SSH par clé = le SEUL accès à une EC2**. Perdre la clé privée, c'est perdre l'accès (pas de « mot de passe oublié » sur un serveur).

Installer la clé publique sur le serveur

Voie confortable — `ssh-copy-id` fait tout (fichier, droits) :

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub deploy@192.168.64.10
```

Méthode manuelle — à connaître : c'est ce que fait `ssh-copy-id`,
et ce que fait **AWS avec votre key pair** au premier boot :

```
cat ~/.ssh/id_ed25519.pub | ssh deploy@192.168.64.10 \  
"mkdir -p ~/.ssh && chmod 700 ~/.ssh && \  
cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"
```

Les **droits sont vérifiés par sshd** — trop ouverts = clé **ignorée**, sans message clair :

Chemin (côté serveur)	Droits exigés
<code>~/.ssh/</code>	<code>700</code> (rwx-----)
<code>~/.ssh/authorized_keys</code>	<code>600</code> (rw-----)

Le confort côté client : `ssh-agent` et

`~/.ssh/config`

`ssh-agent` garde la clé **déchiffrée en mémoire** — la passphrase, une fois par session :

```
eval "$(ssh-agent -s)"      # démarrer l'agent (souvent déjà lancé)
ssh-add ~/.ssh/id_ed25519   # passphrase demandée UNE fois
ssh-add -l                  # lister les clés chargées
```

`~/.ssh/config` (sur le poste, `chmod 600`) nomme vos serveurs — fini le

`ssh deploy@192.168.64.10 -i ~/.ssh/id_ed25519` :

```
Host srv-linux
  HostName 192.168.64.10
  User deploy
  IdentityFile ~/.ssh/id_ed25519
```

Ensuite `ssh srv-linux` suffit — et `scp`/`rsync` profitent des deux réglages.
Avec 40 serveurs (ou 40 instances EC2), ce fichier est votre carnet d'adresses.

Copier des fichiers : **scp** et **rsync**

Les deux passent **dans le canal SSH** (mêmes clés, même config) :

```
scp rapport.txt srv-linux:/tmp/      # un fichier
scp -r projets/ srv-linux:/home/deploy/ # un dossier (récursif)

rsync -av projets/ srv-linux:/home/deploy/projets/
rsync -av --delete projets/ srv-linux:/home/deploy/projets/
```

	scp	rsync
Recopie	tout , à chaque fois	seulement les différences
Reprise après coupure	non	oui
Miroir exact (<code>--delete</code>)	non	oui : supprime en face ce qui a disparu ici
Usage type	dépannage ponctuel	déploiements répétés

`-a` = archive (récursif + droits + dates), `-v` = verbeux.

Piège : `projets/` (avec `/`) copie **le contenu** ; `projets` copie le dossier lui-même.

Tunnel local -L : amener un port distant sur son poste

```

POSTE                                canal SSH chiffré                                VM srv-linux
navigateur → localhost:8080 =====> localhost:80 (nginx)
                                (port 22 seul exposé au réseau)

ssh -L 8080:localhost:80 srv-linux
    |           |           |
    |           |           └─ port CIBLE, sur la machine cible
    |           └─ hôte cible, vu DEPUIS srv-linux (ici : elle-même)
    └─ port ouvert sur MON poste

```

Usage : atteindre un service **non exposé** sans ouvrir le moindre port – tout voyage dans SSH. Vous le referez pour joindre une base **RDS privée** depuis votre poste, via une instance bastion.

Test : `curl http://localhost:8080` sur le poste → réponse du nginx de la VM.

Tunnel distant **-R** : le sens inverse, en une slide

-L **tire** un port distant vers le poste ; **-R** **pousse** un port du poste vers le serveur :

```
ssh -R 9000:localhost:8000 srv-linux
```

- Sur **srv-linux**, le port **9000** mène désormais... au port **8000** de **votre poste**.
- Cas d'usage : montrer une API en cours de développement à quelqu'un qui n'a accès qu'au serveur, déboguer derrière un NAT.
- Vérification côté VM : `curl http://localhost:9000` → répond votre poste.

À retenir pour le choix du sens :

Option	Le port s'ouvre...	...et mène vers
-L	sur mon poste	une machine côté serveur
-R	sur le serveur	une machine côté poste



Exercice 4-1 – Accès SSH par clés et config client (45 min)

Fichier : `exercices/03-cl-linux/exercice-4-1-acces-ssh-par-cles.md`

Depuis votre **poste hôte**, vers la VM `srv-linux` :

- créer une paire **ed25519** ;
- créer l'utilisateur `deploy` sur la VM et installer votre clé publique (`authorized_keys`, droits `700/600`) ;
- écrire l'entrée `Host srv-linux` dans `~/.ssh/config` et vous connecter par `ssh srv-linux` sans mot de passe ;
- copier un dossier avec **scp**, puis avec **rsync -av**, relancer les deux et **comparer** ce qui est réellement transféré.

Bonus : charger la clé dans **ssh-agent** et enchaîner trois connexions sans retaper la passphrase. Difficulté 2/5 – tout est dans les slides.

2. Durcir l'accès

sshd, ufw, fail2ban : fermer ce qui doit l'être

sshd_config et les drop-ins : configurer proprement

La configuration du serveur SSH vit dans `/etc/ssh/sshd_config`... qui commence par :

```
Include /etc/ssh/sshd_config.d/*.conf
```

Bonne pratique (comme pour systemd hier) : **ne pas modifier le fichier principal**, mais poser un **drop-in** — lisible, réversible, il survit aux mises à jour du paquet :

```
sudo nano /etc/ssh/sshd_config.d/90-durcissement.conf
```

```
# /etc/ssh/sshd_config.d/90-durcissement.conf
PermitRootLogin no
PasswordAuthentication no
MaxAuthTries 3
```

Règle sshd : pour chaque directive, **la première valeur lue gagne** — les drop-ins, inclus en tête, priment donc sur le reste. Attention au `50-cloud-init.conf` déjà présent sur les images cloud : vérifiez ce qu'il contient.

Les directives de durcissement à connaître

Directive	Effet	Pourquoi
<code>PermitRootLogin no</code>	root ne peut pas se connecter	on se connecte en utilisateur normal, puis <code>sudo</code> (tracé)
<code>PasswordAuthentication no</code>	clés uniquement	la force brute de mots de passe devient inopérante
<code>MaxAuthTries 3</code>	3 essais par connexion	ralentit les tentatives automatisées
<code>Port 2222</code>	écoute ailleurs que 22	réduit le bruit des scans (pas la sécurité réelle)
<code>AllowUsers deploy ubuntu</code>	liste blanche d'utilisateurs	tout le reste est refusé d'office

Deux mises en garde :

- `Port 2222` impose de suivre partout : `ssh -p 2222`, règle ufw adaptée, entrée `Port 2222` dans `~/.ssh/config`. Ce n'est **pas** du chiffrement en plus.
- Sur vos instances cloud, `PermitRootLogin` et `PasswordAuthentication` sont déjà à `no` par défaut : **l'industrie a tranché**, vous appliquez le même standard.

La règle d'or : **sshd -t** + une session de secours

Une erreur dans **sshd_config** + un restart = **sshd ne redémarre pas** = plus personne n'entre. Sur une VM locale c'est ennuyeux ; sur une EC2, c'est parfois **définitif**. Procédure NON NÉGOCIABLE :

```
sudo sshd -t          # 1. valider la syntaxe : AUCUNE sortie = OK
sudo sshd -T | grep -E 'permitrootlogin|passwordauth|maxauthtries'
                        # 2. voir la config EFFECTIVE après fusion
sudo systemctl restart ssh      # 3. appliquer
```

Et surtout, **la corde de rappel** :

- gardez la session SSH actuelle **OUVERTE** (elle survit au restart) ;
- ouvrez un **NOUVEAU** terminal et testez une connexion fraîche ;
- la nouvelle connexion passe → vous pouvez fermer. Elle échoue → vous réparez depuis la session encore ouverte.

Réflexe à ancrer aujourd'hui : il vous sauvera sur de vraies machines de prod.

ufw : le pare-feu, dans le BON ordre

ufw (Uncomplicated FireWall) : par défaut, une fois activé, **tout ce qui entre est bloqué** sauf exceptions. Or vous êtes connecté... en SSH. Donc :

```
sudo ufw allow OpenSSH      # 1. TOUJOURS autoriser SSH D'ABORD
sudo ufw enable             # 2. SEULEMENT ENSUITE activer
sudo ufw status verbose     # 3. vérifier
```

```
Status: active
Default: deny (incoming), allow (outgoing), disabled (routed)
To      Action    From
--      -
OpenSSH ALLOW IN  Anywhere
```

Inverser 1 et 2 = **couper la branche sur laquelle vous êtes assis** :

la session tombe, et sans console physique, plus personne n'entre.

OpenSSH est un profil d'application (`sudo ufw app list`) — équivalent à `22/tcp`.

ufw au quotidien — et son jumeau AWS

```
sudo ufw allow 80/tcp      # ouvrir HTTP (nginx, tout à l'heure)
sudo ufw status numbered  # règles avec numéros
sudo ufw delete 2          # supprimer la règle n°2
sudo ufw delete allow 80/tcp # ou par sa définition
sudo ufw disable           # tout désactiver (diagnostic)
```

Principes à retenir :

- ouvrez **le strict nécessaire** : ici 22 (SSH) et 80 (HTTP), rien d'autre ;
- `status verbose` avant et après chaque changement — pas de règle « au cas où » ;
- un port fermé par ufw : `curl` échoue mais `ping` répond (souvenez-vous du J3).

Le security group AWS, c'est le ufw géré par AWS : mêmes questions (quel port ? quelle source ?), même politique « tout est fermé sauf... » — simplement appliqué **devant** l'instance au lieu de dedans. Vous ferez les deux : SG côté cloud, ufw côté OS = défense en profondeur.

fail2ban — démonstration formateur

Que faire des IP qui tentent 500 mots de passe par heure ? **fail2ban** lit les journaux d'authentification et **bannit temporairement** (via le pare-feu) les IP qui accumulent les échecs. Principe de la **jail** `sshd` :

```
journal auth —> filtre (regex échecs SSH) —> seuil atteint ?
                        | oui
                        ban de l'IP pendant 10 min
```

```
sudo apt install -y fail2ban
sudo systemctl enable --now fail2ban
sudo fail2ban-client status      # jails actives : sshd
sudo fail2ban-client status sshd # échecs vus, IP bannies
```

Démonstration au vidéoprojecteur : échecs volontaires, IP bannie, dé-ban.

Pas d'exercice dédié — vous le relirez au bonus de l'exercice 4-2.

Les couches de défense de votre serveur



Aucune couche n'est suffisante seule ; ensemble, elles éliminent l'essentiel du risque. Sur AWS : **security group** devant, puis exactement ces trois couches **dans** l'instance.

Exercice 4-2 – Durcissement sshd et pare-feu ufw (45 min)

Fichier : `exercices/03-cl-linux/exercice-4-2-durcissement-sshd-ufw.md`

Sur votre VM `srv-linux` (avec l'accès par clé de l'exercice 4-1) :

- écrire le drop-in `/etc/ssh/sshd_config.d/90-durcissement.conf` :
`PermitRootLogin no`, `PasswordAuthentication no`, `MaxAuthTries 3` ;
- valider avec `sudo sshd -t` **AVANT** le restart, appliquer en gardant **une session ouverte** (la corde de rappel) ;
- activer **ufw** : `allow OpenSSH` puis `allow 80/tcp` **avant** `enable`, contrôler avec `status verbose` ;
- prouver qu'une connexion par mot de passe est désormais refusée.

Bonus : passer sshd sur le **port 2222** (et en mesurer toutes les conséquences), puis lire l'état de la jail **fail2ban** (`sudo fail2ban-client status sshd`).

Difficulté 3/5 – c'est l'exercice où l'on apprend à ne pas s'enfermer dehors.

3. Supervision de base

Lire l'état de santé d'un serveur en 60 secondes

uptime et la charge moyenne

```
uptime
# 14:02:11 up 3 days, 2:47, 2 users, load average: 0.42, 0.31, 0.18
```

Le **load average** = nombre moyen de processus **en attente d'exécution** (CPU occupé ou I/O bloquantes) sur **1, 5 et 15 minutes**.

Lecture : comparez à votre nombre de CPU (`nproc` → **2** sur la VM) :

Charge (VM à 2 CPU)	Interprétation
0.42	machine tranquille
2.0	les 2 CPU sont pleinement occupés
6.0	~4 processus attendent en permanence : saturation

Les trois valeurs donnent la **tendance** : `4.0, 1.0, 0.2` = pic en cours ;
`0.3, 2.0, 4.0` = ça se calme. Premier chiffre à regarder quand « c'est lent ».

free -h : où en est la mémoire ?

```
free -h
#                total    used    free   shared  buff/cache   available
# Mem:           3.8Gi    612Mi    1.9Gi    1.0Mi      1.4Gi      3.0Gi
# Swap:           0B         0B         0B
```

- **available** = la colonne qui compte : ce que le système peut réellement donner aux applications (en récupérant le cache au besoin).
- **free** bas n'est **pas** un problème : Linux utilise la RAM libre comme **cache disque** (**buff/cache**) — c'est une qualité, pas une fuite.
- **Swap used qui grimpe** = vrai signal d'alerte : la RAM ne suffit plus, les performances s'effondrent.

Réflexe : **available** faible **et** swap actif → chercher le processus gourmand (**htop**, tri par mémoire — vu au J2).

vmstat 1 5 : le tableau de bord en colonnes

```
vmstat 1 5          # une mesure par seconde, 5 fois
#procs  -----memory----- --swap--  ----io----  -system--  -----cpu-----
#  r  b   swpd free ...   si  so   bi   bo   in  cs  us sy id wa st
#  1  0       0  1.9G       0   0    3   12  120  240   2  1 97  0  0
```

Les colonnes utiles (ignorez le reste pour l'instant) :

Colonne	Sens	Alerte si...
r	processus prêts, en attente de CPU	durablement > nb CPU
si / so	swap in / out (pages/s)	≠ 0 en continu → manque de RAM
us / sy	% CPU applicatif / noyau	us haut = l'appli calcule
wa	% CPU en attente d'I/O disque	> 20 % → le disque est le goulot
id	% CPU inactif	proche de 0 = saturation

La **première ligne** est une moyenne depuis le boot : lisez les suivantes.

iostat, df -h : disques sous surveillance

`iostat` (paquet `sysstat`) détaille l'activité disque – lecture légère ici :

```
sudo apt install -y sysstat
iostat -xz 1 3          # %util proche de 100 = disque saturé, w/s, r/s
```

Et le **réflexe quotidien** que vous avez déjà (J3) :

```
df -h                  # un système de fichiers à 100 % = pannes en cascade
```

Un disque plein casse **tout** : journaux muets, base qui refuse d'écrire, service qui tombe – panne n°1 des serveurs laissés sans surveillance.

La routine des 60 secondes : `uptime` → `free -h` → `df -h` → (`vmstat` si doute).

Demain, dans le cloud, CloudWatch fera ces mesures en dashboard – mais devant un incident, c'est TOUJOURS cette routine que vous taperez en premier.

4. Fil rouge : StockLine part en production

Tout le bloc CL-LINUX dans un seul déploiement

L'architecture cible

```
Internet/hôte —80—> nginx (reverse proxy) —127.0.0.1:8000—> uvicorn (StockLine)
                                                                |
                                                                /opt/stockline/data/stockline.db
```

- **nginx** seul est exposé (port 80) : il **proxifie** vers uvicorn, qui n'écoute que sur `127.0.0.1:8000` — jamais accessible de l'extérieur.
- **uvicorn** exécute l'API FastAPI (StockLine « lite » du bloc CL-PYTHON, version SQLite), sous un **utilisateur système dédié**, piloté par **systemd**.
- Les **données** vivent dans `/opt/stockline/data`, sauvegardées chaque nuit.

C'est le schéma standard des déploiements Python — et **mot pour mot le CL-TP1 noté** (avec PostgreSQL en plus), puis l'user data de vos instances.

Étapes 1-2 – Transférer le code, préparer le terrain

Transfert depuis le poste (le cours SSH du matin sert immédiatement) :

```
rsync -av code/03-cl-linux/stockline-lite/ srv-linux:/tmp/stockline-app/
```

Sur la VM – utilisateur système + arborescence :

```
sudo adduser --system --group --home /opt/stockline \  
--shell /usr/sbin/nologin stockline  
sudo mkdir -p /opt/stockline/app /opt/stockline/data  
sudo cp -r /tmp/stockline-app/. /opt/stockline/app/
```

- `--system` : utilisateur **technique** (pas de mot de passe, UID réservé) ;
- `--shell /usr/sbin/nologin` : **personne** ne peut ouvrir de session avec – si l'API est compromise, l'attaquant hérite d'un compte sans shell ni sudo.
- Arborescence : `app/` (le code), `venv/` (Python isolé), `data/` (la base).

Étape 3 — venv, dépendances, test manuel

```
sudo apt update && sudo apt install -y python3-venv nginx
sudo python3 -m venv /opt/stockline/venv
sudo /opt/stockline/venv/bin/pip install -r /opt/stockline/app/requirements.txt
sudo chown -R stockline:stockline /opt/stockline
```

Toujours tester à la main AVANT d'écrire l'unité systemd — sinon vous déboguerez deux problèmes à la fois :

```
cd /opt/stockline/app
sudo -u stockline env STOCKLINE_DB=/opt/stockline/data/stockline.db \
/opt/stockline/venv/bin/uvicorn main:app --host 127.0.0.1 --port 8000
# autre terminal : curl http://127.0.0.1:8000/sante -> {"statut":"ok"}
# puis Ctrl+C pour arrêter le test
```

STOCKLINE_DB : l'API lit le chemin de sa base dans cette variable d'environnement (défaut **./stockline.db** en développement).

Étape 4 — `stockline.service` : systemd prend le relais

```
# /etc/systemd/system/stockline.service
[Unit]
Description=API StockLine (uvicorn)
After=network.target

[Service]
User=stockline
WorkingDirectory=/opt/stockline/app
Environment=STOCKLINE_DB=/opt/stockline/data/stockline.db
ExecStart=/opt/stockline/venv/bin/uvicorn main:app --host 127.0.0.1 --port 8000
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload && sudo systemctl enable --now stockline
systemctl status stockline && journalctl -u stockline -n 20
```

Tout vient du J2 : `Restart=on-failure` relance l'API si elle plante,
`enable` la démarre à chaque boot. Fini le processus lancé à la main.

Étape 5 — nginx en reverse proxy

```
# /etc/nginx/sites-available/stockline.conf
server {
    listen 80;
    server_name _;
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/stockline.conf /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
```

Pourquoi nginx devant uvicorn ? Port 80 (privilégié) sans lancer l'API en root, uvicorn jamais exposé, gestion des connexions lentes, fichiers statiques — et bientôt le **TLS**. `nginx -t` d'abord, toujours (même logique que `sshd -t`).

Étape 6 — Vérifier aux 3 niveaux (le réflexe qui localise la panne)

```
# 1. Sur la VM : l'API répond-elle en direct ?
curl http://127.0.0.1:8000/sante

# 2. Sur la VM : nginx proxifie-t-il correctement ?
curl http://localhost/sante

# 3. Depuis le POSTE : la VM est-elle joignable de l'extérieur ?
curl http://192.168.64.10/sante
```

Chaque niveau **isole une couche** :

Échec au niveau...	Le coupable est...	Premier réflexe
1 (port 8000)	l'API / le service	<code>journalctl -u stockline</code>
2 (port 80 local)	nginx / sa conf	<code>nginx -t</code> , <code>journalctl -u nginx</code>
3 (depuis l'hôte)	réseau / pare-feu	<code>sudo ufw status</code> , IP de la VM

Puis un vrai test métier : `curl -X POST http://localhost/produits -H "Content-Type: application/json" -d '{"nom":"clavier","seuil_alerte":5}'`

Étapes 7-8 — Sauvegarder les données, fermer les ports

Le `backup.sh` d'hier (démon 3-1) resservi tel quel, nouvelles cibles :

```
# depuis le poste :  
rsync -av code/03-cl-linux/backup.sh srv-linux:/tmp/  
# sur la VM :  
sudo install -m 755 /tmp/backup.sh /usr/local/bin/backup.sh  
sudo /usr/local/bin/backup.sh /opt/stockline/data /var/backups/stockline  
ls /var/backups/stockline # data-AAAAMMJJ-HHMMSS.tar.gz
```

Planification : `backup.service` (oneshot) + `backup.timer`
(`OnCalendar=*-*-* 03:00:00`, `Persistent=true`) → `systemctl list-timers`.

Enfin, le durcissement de ce matin s'applique à la prod :

```
sudo ufw allow OpenSSH && sudo ufw allow 80/tcp  
sudo ufw enable && sudo ufw status verbose
```

Automatiser (script + timer), **superviser** (curl /sante, journalctl),
sécuriser (user système, ufw) : la devise du cursus, en une machine.

Démo 4-1 — Le déploiement complet, en direct (45 min)

Fichier : `demos/03-cl-linux/demo-4-1-deploiement-stockline.md`

Je déroule **tout le déploiement en accéléré** au vidéoprojecteur — vous observez, vous questionnez, vous refaites juste après en mini-TP :

1. `rsync` du code depuis le poste ;
2. utilisateur système `stockline` + arborescence `/opt/stockline` ;
3. venv, dépendances, **test uvicorn manuel** ;
4. `stockline.service` → `enable --now` ;
5. nginx en reverse proxy → `nginx -t` → reload ;
6. `curl /sante` **aux 3 niveaux** ;
7. panne simulée : je **tue uvicorn** → `Restart=on-failure` le relance sous vos yeux (comptez les secondes).

Notez les points où **vous** feriez une pause-vérification : ce sont les points de contrôle du mini-TP.

Mini-TP — Votre première mise en production (~2 h, non noté)

Fichier : `tp/03-cl-linux/tp-mini-serveur.md`

À votre tour, **en autonomie guidée** : les 9 étapes numérotées, chacune avec son **point de contrôle** (une commande, une sortie attendue). Dépannage courant fourni en dépliants (`502 Bad Gateway`, port déjà pris, permission denied sur la base, service en échec).

- **Non noté**, checklist d'auto-validation en fin de sujet.
- C'est la **répétition générale du CL-TP1** : dans quelques jours vous referez ce déploiement **noté**, avec PostgreSQL et un script Python d'exploitation.
- Puis ce sera l'**user data** de vos instances EC2 : les mêmes commandes, exécutées automatiquement au premier boot.

⚠ **Ne supprimez PAS la VM à la fin** : elle resservira au CL-TP1.

Un `multipass stop srv-linux` suffit pour ce soir.

Récap du bloc CL-LINUX – 4 jours en une slide

Jour	Cap	Ce que vous savez faire désormais
J1	Prendre la main	naviguer, manipuler fichiers, <code>man</code> , nano, utilisateurs, <code>sudo</code> , droits <code>rwX</code> /octal, umask
J2	Faire tourner	processus/signaux, <code>systemctl</code> , écrire une unité, APT, <code>journalctl</code> , logrotate
J3	Automatiser	scripts bash (arguments, <code>\$?</code> , boucles), grep/awk/sed, cron + timers, <code>df</code> / <code>du</code> /fstab, <code>ip</code> / <code>ss</code> / <code>curl</code> / <code>dig</code>
J4	Ouvrir, sécuriser, déployer	SSH par clés, rsync, tunnels, sshd durci, ufw, fail2ban, supervision, StockLine en prod

La VM `srv-linux` est passée de « machine vide » à **serveur de production sauvegardé et durci** – uniquement au clavier, sans jamais cliquer.

Automatiser, superviser, sécuriser : vous l'avez fait pour de vrai.

Toutes les commandes du bloc : `cheatsheet/03-cl-linux.md`.



Quiz — questions 1 et 2

Question 1

Pourquoi préférer l'authentification par clé SSH au mot de passe ?

Question 2

Où doit se trouver la clé publique sur le serveur, et avec quels droits ?



Quiz — questions 3 et 4

Question 3

Que fait `ssh -L 8080:localhost:80 srv-linux` ?

Question 4

Citez trois directives de durcissement vues dans `sshd_config`.



Quiz — questions 5 et 6

Question 5

Quelle est la différence entre scp et rsync pour copier un dossier ?

Question 6

Pourquoi lancer `sudo ufw allow OpenSSH` AVANT `sudo ufw enable` ?



Quiz — questions 7 et 8

Question 7

Quel est le rôle de fail2ban ?

Question 8

Dans notre déploiement, pourquoi placer nginx devant uvicorn ?



Quiz — questions 9 et 10

Question 9

Quelle commande vérifie en local que l'API StockLine répond ?

Question 10

Après modification de stockline.service, quelles deux commandes enchaîner ?

Bravo — et à demain !

Cap sur CL-RÉSEAU (jours 10 à 13)

Vous savez administrer **LA** machine ; il faut maintenant comprendre ce qui circule **ENTRE** les machines. Dès demain : modèles OSI et TCP/IP, adressage IP et calcul de sous-réseaux **à la main** — la brique de tous vos futurs VPC. **Gardez précieusement votre VM `srv-linux`** (un simple `multipass stop`, surtout pas de delete) : elle resservira au CL-TP1 noté.